



29. One-vs-One and One-vs-All: Multi-Class Classification

One for All, and All for One

Previous Section:

 [28. Support Vector Machines](#)

Contents:

[1. One-vs-One Approach \(OvO\)](#)

[2. One-vs-All Approach \(OvA\)](#)

[3. Final Insight: OvO vs OvA \(Comparison Table\)](#)

[4. Support Vector Machine: One-vs-One and One-vs-All approach with Python](#)

[Summary](#)

[References](#)

Every model has its comfort zone. Some models think in probabilities. Some think in boundaries. Some think in decisions. But many of them, share one thing in common: They are naturally built for **two classes**.

Take a moment and think about it.

- Models like Decision Tree, Random Forest, and Boosting methods make decisions based on drawing depth-level trees.
- A model like Logistic Regression draws a line between two groups.
- Support Vector Machines? They go even further—finding the *best possible boundary* that separates two worlds.

- Even Naive Bayes, at its core, answer a simple question: *"Is it this... or not?"*

Binary thinking. And honestly? That works beautifully: Spam vs Not Spam; Fraud vs Legitimate; Disease vs No Disease → Common problems. Clean. Clear. Effective.

But here's the problem. The real world is rarely that simple.

- You don't just classify emails. → You classify **topics**.
- You don't just detect disease. → You identify **types of diseases**.
- You don't just recognize an image. → You decide whether it's a ***cat, dog, horse... or something else entirely***.

And suddenly. Two classes are not enough.

So what do we do? Do we redesign every model from scratch to handle multiple classes? Well we *could*. But that would be complicated. Inefficient. And honestly unnecessary.

Because there's a much smarter idea. Instead of forcing a model to handle everything at once. We **break the problem down**. We take a multi-class problem. And turn it into multiple **binary problems**. Simple pieces. Familiar structure. Same models.

Think of it like this. You're trying to identify one person in a group of many. You could try to recognize everyone at once. Or you could ask simpler questions:

- *"Is it this person?" → "If not... is it that one?" → "Or maybe between these two?"*

And step by step. You narrow it down. That's exactly the philosophy behind multi-class strategies.

Among all the approaches that follow this idea, two stand out. Two strategies that are simple, powerful, and widely used: **One-vs-All (OvA)** and **One-vs-One (OvO)**

They take the same problem. But look at it from two completely different perspectives.

- One asks: *"Can this class beat all others?"*; The other asks: *"Which class wins in a head-to-head battle?"*

And from these simple ideas. We can turn binary classifiers into full multi-class systems.

In the next sections. We'll go through both strategies in detail.

1. One-vs-One Approach (OvO)

Let's approach this the way we usually understand things best, through people. Imagine a tournament. Not a group discussion. Not everyone speaking at once. But a series of **one-on-one battles**.

- Two participants step forward. They compete. A winner is decided.
- Then another pair steps in. Another decision. Another winner.
- And by the end of the tournament. You don't need everyone to fight at once. You just need enough **pairwise comparisons** to figure out who stands above the rest.

→ That is exactly how the **One-vs-One (OvO)** strategy thinks.

Instead of trying to separate all classes in a single step, OvO breaks the problem into the smallest possible pieces: **Two classes at a time**

For every possible pair of classes, we train a **separate binary classifier**. No distractions. No interference from other classes. Just a clean, focused decision: *"Between these two... which one is it?"*

Let's make this concrete. Suppose we have four classes: A, B, C, D

→ OvO will not try to solve this in one model. Instead, it creates classifiers for every possible pair: $(A \text{ vs } B); (A \text{ vs } C); (A \text{ vs } D); (B \text{ vs } C); (B \text{ vs } D); (C \text{ vs } D)$

→ A total of **6 classifiers**

Each one is responsible for a **very specific question**. And here's something important: These pairs are **unordered**. $(A \text{ vs } B)$ is the same as $(B \text{ vs } A)$. So we don't duplicate effort.

Now, look at what happens during training. Each classifier doesn't see the full dataset. Instead, the dataset is **filtered**

If we are training $(A \text{ vs } C)$:

- We keep only samples from class A and class C
- Everything else $(B, D, \dots)$ is ignored

So each model works with a **smaller** dataset, a **cleaner** problem, and often a more **balanced** distribution → This makes learning easier. No noise from

unrelated classes. No confusion from overlapping regions. Just two classes facing each other.

And if we count how many classifiers we need? For n **classes**, the total number is:

$$\frac{n(n-1)}{2}$$

Because we are forming all possible unique pairs.

Now comes the interesting part: **Prediction** → When a new data point arrives. We don't rely on just one model. We ask **all classifiers**.

Each classifier makes a decision between its two classes. So for our example:

- $(A \text{ vs } B) \rightarrow \text{votes } A$
- $(A \text{ vs } C) \rightarrow \text{votes } C$
- $(A \text{ vs } D) \rightarrow \text{votes } A$
- $(B \text{ vs } C) \rightarrow \text{votes } C$
- $(B \text{ vs } D) \rightarrow \text{votes } D$
- $(C \text{ vs } D) \rightarrow \text{votes } C$

Now we collect the votes: $A \rightarrow 2$ votes; $C \rightarrow 3$ votes; $D \rightarrow 1$ vote → Final prediction: C

This is called a **majority voting mechanism (just like Random Forest)**. Each classifier contributes a small opinion. And the final decision emerges from the collective.

Characteristics of One-vs-One

Now that we see how it works, we can understand its nature.

- **Strong when classes are well-separated:** Because each classifier focuses on only two classes, it can learn very precise boundaries. If the classes are clearly distinguishable, OvO performs extremely well.
- **Simpler subproblems:** Each model deals with less data, fewer classes, cleaner structure → This often leads to better local decisions.

- **More balanced training:** Since we only keep two classes at a time, class imbalance is usually reduced. This helps many algorithms perform more reliably.
- **But computationally expensive:** There is a trade-off. As the number of classes grows. The number of classifiers increases rapidly:
 - 3 classes → 3 models
 - 5 classes → 10 models
 - 10 classes → 45 models
 - 100 classes → 4950 models
 → That's a lot.

Training becomes expensive. Storage becomes heavier. Prediction becomes slower.

But this is the essence of One-vs-One:

Break everything into pairwise battles → Let each model make a simple decision → Combine them through voting

It doesn't try to solve the problem all at once. Instead, it solves many small problems. And from those small answers. It builds one final decision.

2. One-vs-All Approach (OvA)

Let's go back to people again. But this time... not a tournament. Instead, imagine an interview. One person stands in front of a panel.

- And the question is simple: *"Are you this person... or not?"*
- Then another person steps forward. Same question. Different identity: *"Are you that person... or not?"*
- And we repeat this process. Until every possible identity has been checked.

That is how **One-vs-All (OvA)** thinks. Instead of comparing classes in pairs. OvA takes a different perspective: **One class vs everything else**

Let's make this concrete. Suppose we still have four classes: A, B, C, D

OvA will build the following classifiers:

- Classifier 1: A vs (B, C, D)

- Classifier 2: B vs (A, C, D)
- Classifier 3: C vs (A, B, D)
- Classifier 4: D vs (A, B, C)

So instead of pairwise battles. Each classifier answers a **single focused question**:

- *"Is this sample class A?"* → *"Is this sample class B?"* → *"Is this sample class C?"* → *"Is this sample class D?"*

Now look at what happens during training. Each model is trained with:

- One class labeled as **positive (+1)**
- All other classes grouped as **negative (−1)**

So for (A vs Rest): $A \rightarrow$ positive; $B, C, D \rightarrow$ negative

This creates a very different learning problem compared to OvO. Instead of a clean two-class comparison. We now have one class against a **mixed group of everything else**

And how many models do we need? For n **classes**, we train exactly n **classifiers** → Much simpler than OvO.

Now comes prediction. When a new sample arrives. We don't vote like in OvO. Instead, each classifier produces a **score**. Not just a label—but a measure of confidence: *"How likely is this sample to belong to my class?"*

So we might get something like:

- Classifier $A \rightarrow 0.2$
- Classifier $B \rightarrow 0.8$
- Classifier $C \rightarrow 0.4$
- Classifier $D \rightarrow 0.1$

And then we simply choose: The class with the **highest score** → Final prediction: B

This is very similar to how **Logistic Regression** works. Not just yes/no decisions. But confidence-based reasoning.

Characteristics of One-vs-All

Now that we understand the mechanics, let's look at its behavior.

- **Fewer models, simpler structure:** OvA only requires: n classifiers, which makes it much lighter compared to OvO. Less training time. Less storage. Faster prediction.
- **Easy to implement and scale:** Because of its simplicity, OvA is widely used in practice. Even when the number of classes becomes large, it remains manageable. This makes it a very practical choice for real-world systems.
- **But class imbalance becomes an issue:** Here's the trade-off. Each classifier sees: One class vs many others. So the data becomes **imbalanced**:
 - Positive samples → few
 - Negative samples → many
 - This can make learning harder. The model might become biased toward the negative class. Especially if the target class is small.
- **Decision boundaries can be less precise:** Since the negative class is a mixture of multiple classes. The boundary is not always clean. Unlike OvO (which compares two specific classes), OvA must separate one class from a **complex combination** of others.

3. Final Insight: OvO vs OvA (Comparison Table)

Aspect	One-vs-One (OvO)	One-vs-All (OvA)
Core Idea	Compare classes in pairs	Compare one class against all others
Strategy	Many small, precise decisions	Fewer, broader decisions
Number of Classifiers	$\frac{n(n-1)}{2}$	n
Training Data per Model	Only samples from 2 classes	Full dataset (1 vs rest)
Data Balance	More balanced (2 classes only)	Often imbalanced (1 vs many)
Decision Mechanism	Majority voting across classifiers	Highest confidence/score wins

Aspect	One-vs-One (OvO)	One-vs-All (OvA)
Scalability (Many Classes)	Poor (models grow rapidly)	Good (linear growth)
Performance (Few Classes)	Very strong, precise boundaries	Good, but less fine-grained
Handling Imbalance	Naturally better	Needs careful handling
Computational Cost	High (many models)	Lower (fewer models)
Interpretability	More complex (many votes)	Simpler (direct score comparison)
Typical Use Case	Small to medium class problems	Large-scale classification problems
Final Takeaway		

Situation	Preferred Approach
Few classes, need precision	OvO
Many classes, need efficiency	OvA
Balanced dataset	OvO
Imbalanced dataset	OvO (safer) or OvA with adjustments

In the end, Multi-class classification is not about changing the model. It's about choosing the **right strategy**. And with OvO and OvA, even a simple binary classifier can scale up, and make sense of a much more complex world

4. Support Vector Machine: One-vs-One and One-vs-All approach with Python

Now let's bring everything together. Because up until this point, OvO and OvA might feel like strategies floating in theory. Nice ideas. Clean concepts. But the real question is:

Do they actually work in practice? → And more importantly: Which model fits them naturally?

Let's start with something familiar. **Support Vector Machines (SVMs)**. You've seen them before. They don't try to classify everything perfectly. They don't chase every single data point.

Instead, they draw a boundary. A very careful one. One that keeps classes as far apart as possible.

That's their philosophy. But here's the key detail: An SVM is, by nature, a **binary classifier**. It knows how to separate **two classes**. Nothing more.

So what happens when we give it three classes? Four? Ten? Does it break? Not at all. Because instead of changing the SVM. We change the **strategy around it**. And this is where OvO and OvA come in.

Why SVM Fits Perfectly with OvO and OvA? This is not just about having a parameter in a library. Or we just randomly use SVM. There's a deeper reason. SVM works best when:

- The boundary between classes is **clear**
- The margin between them can be **maximized**

Now think about OvO. Each classifier only sees **two classes**. No noise. No interference. Just a clean separation problem → This is exactly what SVM loves. → That's why OvO + SVM is often a very powerful combination.

What about OvA? Now the problem becomes: One class vs all others → This is harder. Because the "other" class is actually a mixture of many classes. The boundary becomes more complex. But still, SVM can handle it. Because even in complex spaces (especially with kernels), it can still find a separating boundary.

So in short:

- **OvO + SVM** → Clean, precise, often very strong
- **OvA + SVM** → Simpler, scalable, still effective

Scikit-Learn: Making It Practical

In real-world applications, we don't implement OvO and OvA from scratch. Libraries like **Scikit-learn** already handle it for us. And here's something interesting:

The `SVC` class (Support Vector Classifier) uses **One-vs-One by default**. But we can also switch to OvA if needed. So instead of worrying about the mechanics. We focus on Data → Model configuration → Evaluation.

And there's one more important detail. Scikit-learn also provides dedicated wrappers: `OneVsOneClassifier` and `OneVsRestClassifier`. But these are not

standalone models. They don't learn anything by themselves. Instead, they act as **strategies** that wrap around another base model

For example:

- `OneVsOneClassifier` + Logistic Regression
- `OneVsRestClassifier` + SVM
- `OneVsRestClassifier` + Decision Tree

So rather than being classifiers. They are **adapters**. They take a binary model, and extend it into a multi-class system using OvO or OvA logic.

Which reinforces the main idea once again: We are not changing the model. We are changing **how the model is used**.

The Dataset: Glass Identification

To make things concrete, we will use the **Glass Identification Dataset**. This is a classic multi-class classification problem. Each sample represents a type of glass, based on its chemical composition. And the goal? Predict the **type of glass**

Not two classes. Not even three. Multiple categories. Which makes it perfect for testing: OvO vs OvA with SVM

Here, we implement SVM using Scikit-learn with OvO and OvA strategies on the Glass dataset.

[glass.csv](#)

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv('glass.csv')
target = 'Type'
features = df.columns.drop(target)

# 80% training and 20% test
```

```
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
X_train = train_df[features]
X_test = test_df[features]
y_train = train_df[target]
y_test = test_df[target]
```

```
# Multi-Class Classification: One-vs-One approach
# Model: Support Vector Machine
from sklearn.svm import SVC
from sklearn.multiclass import OneVsOneClassifier
from sklearn.metrics import accuracy_score

# Train an SVC model
model = SVC(kernel='linear')
# One-vs-One (OvO) approach
ovo_classifier = OneVsOneClassifier(model)
ovo_classifier.fit(X_train, y_train)
y_pred_ovo = ovo_classifier.predict(X_test)
accuracy_ovo = accuracy_score(y_test, y_pred_ovo)
print(f"One-vs-One Accuracy:", accuracy_ovo)
```

One-vs-One Accuracy: 0.7674418604651163

```

# Multi-Class Classification: One-vs-All approach
# Model: Support Vector Machine
from sklearn.svm import SVC
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import accuracy_score

# Train an SVC model
model = SVC(kernel='linear')
# One-vs-All (OvA) approach
ova_classifier = OneVsRestClassifier(model)
ova_classifier.fit(X_train, y_train)
y_pred_ova = ova_classifier.predict(X_test)
accuracy_ova = accuracy_score(y_test, y_pred_ova)
print(f"One-vs-All Accuracy:", accuracy_ova)

```

One-vs-All Accuracy: 0.5581395348837209

The accuracy is quite subpar. But this is the reality (*your reality* → *remember whose song this is??*). Not every dataset is clean. Not every model is perfect. And not every result is impressive.

And that's okay. Because this is where learning actually begins: Not when accuracy is high. But when you start asking: *Why isn't it?*

We could also plot the confusion matrices (*Because why not??*)

```

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

conf_matrix_ovo = confusion_matrix(y_test, y_pred_ovo)
conf_matrix_ova = confusion_matrix(y_test, y_pred_ova)

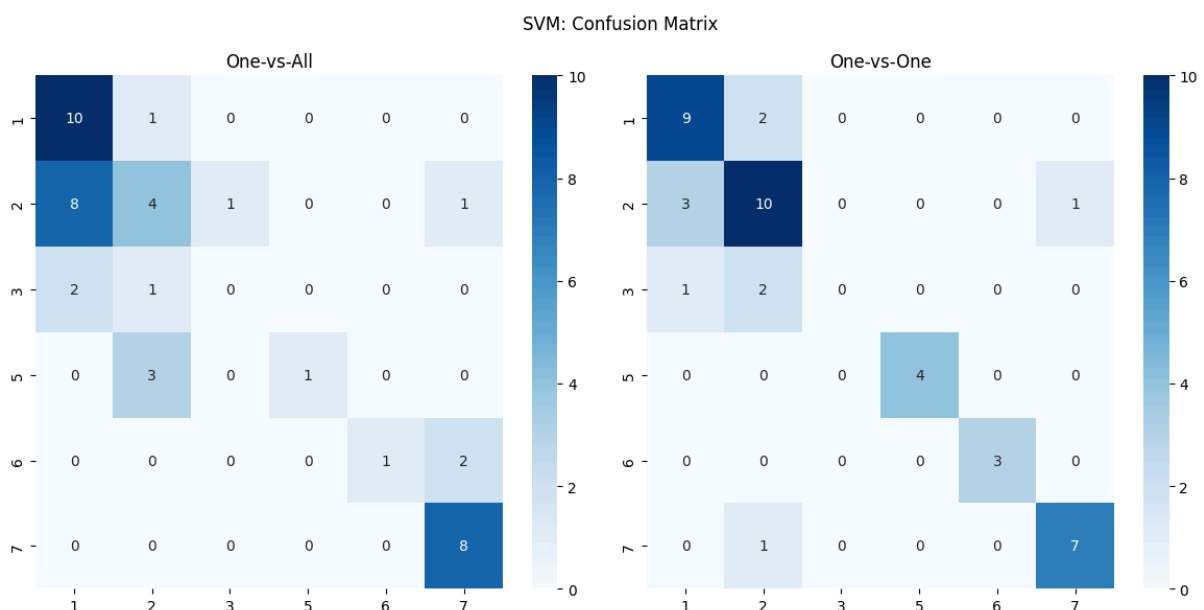
unique_labels = df[target].unique().tolist()
# Plot the confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
sns.heatmap(conf_matrix_ova, ax=axes[0], annot=True, fmt="d",
cmap="Blues",

```

```

xticklabels=unique_labels, yticklabels=unique_labels)
axes[0].set_title("One-vs-All")
sns.heatmap(conf_matrix_ovo, ax=axes[1], annot=True, fmt
="d",
cmap="Blues",
xticklabels=unique_labels, yticklabels=unique_labels)
axes[1].set_title("One-vs-One")
plt.suptitle("SVM: Confusion Matrix")
plt.tight_layout()
plt.show()

```



Summary

At first, multi-class classification seems like a completely different problem. More classes. More complexity. More things to separate. But OvO and OvA reveal something important:

■ We don't need to reinvent the model. → We just need to rethink the strategy.

Both approaches follow the same core idea: Break a complex multi-class problem into **simpler binary tasks** → But the way they do it, changes everything.

One-vs-One (OvO) takes a precise approach: **It compares classes pair by pair**
→ **Builds many small, focused classifiers** → **And lets them vote to decide the final answer**

This leads to:

- Cleaner decision boundaries
- Better handling of balanced comparisons
- But higher computational cost as classes grow

One-vs-All (OvA) takes a broader view: **It compares one class against all others** → **Builds fewer models** → **And selects the class with the highest confidence**

This makes it:

- Simpler and more scalable
- Easier to implement
- But more sensitive to class imbalance

So in the end. There is no universal winner. Only trade-offs.

- Precision vs Simplicity
- Many small decisions vs Few global ones
- Computational cost vs Scalability

And that's the real lesson here: Multi-class classification is not about building a new model. It's about choosing the right **perspective**. Because with OvO and OvA, even a binary classifier can learn to navigate a world with many possibilities.

References

<https://www.nb-data.com/p/one-vs-all-vs-one-vs-one-which-multi>

<https://machinelearningmastery.com/one-vs-rest-and-one-vs-one-for-multi-class-classification/>

Next Section:

 **30. Oversampling and Undersampling: Handling Imbalanced Data**

